Universitetet
i Sørøst-Norge

www.usn.no

PB1090 Objekt orientert programmering

Obligatorisk oppgave 3

Universitetet
i Sørøst-Norge

Fakultet for teknologi, naturvitenskap og maritime fag

Campus Vestfold

Innlevering

Du skal levere en eller flere Java fil(er):

- Oblig3SudokuSolver_<`student_nummer`>.java

NB. <`student_nummer`> skal skiftes ut med ditt studentnummer uten <>.

Pass på at filnavn og klassenavn blir riktig slik at innleveringsfilen vil kompilere og kan kjøres ved testing av Studentassistent. Ikke ta med .class filene (binære bytecode filer). Dersom du bruker Linux terminal, kan de slettes med kommandoen: `rm *.class`

Innleveringsfrist: 27. mars kl. 23:59 på Canvas siden til PB1090 under «Oppgaver».

Viktig om egen innsats og bruk av KI

Vi har lagt opp oppgaven slik at den skal være mulig å løse uten bruk av KI verktøy eller andre hjelpemidler. Det er viktig at du prøver selv og forstår koden du leverer.

Husk at KI eller andre hjelpemidler ikke vil være tilgjengelig under eksamen.

Sjekk også USN sine retningslinjer for bruk av kunstig intelligens (KI) ved eksamen og student-oppgaver: Retningslinjer for bruk av KI (USN)

Bakgrunn - Sudoku

Sudoku, også kalt **su doku**, er et tallspill som går ut på å plassere sifre i rutemønstre. Det klassiske og opprinnelige rutemønsteret består av 9×9 ruter som igjen er delt inn i ni mindre bokser på 3×3 ruter. Oppgaven går ut på å plassere sifrene 1 til 9 slik at alle de loddrette og vannrette radene inneholder sifrene 1 til 9 én gang (slik at også hver 3×3-boks inneholder alle sifrene én gang).

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Figur 1: Eksempel på et Sudoku-brett

NB
Dette er vanlig Sudoku uten diagonalen fra (0,0) til (8,8) og (8,0) til (0,8), som kalles X-Sudoku.

Vanlig måte å spille på:
Sudoku har i mange år vært et populært spill i ukeblader og aviser. Slik som kryssord var i "gode

gamle dager". Løsning av Sudoku gjøres som regel med "prøving og feiling". Det vil si man prøver å sette ut tall i en celle det er ledig, men man må passe på at man ikke setter dem slik at Sudoku brett blir uløselig eller ugyldig.

På en datamaskin:
På samme måte som med penn og papir, kan man løse Sudoku på en PC. For eksempel i en nettleser (Chrome/Firefox/Edge), der man bruker musepeker/ og eller tastatur for å sette ut tall. Prøv selv: Spill Sudoku

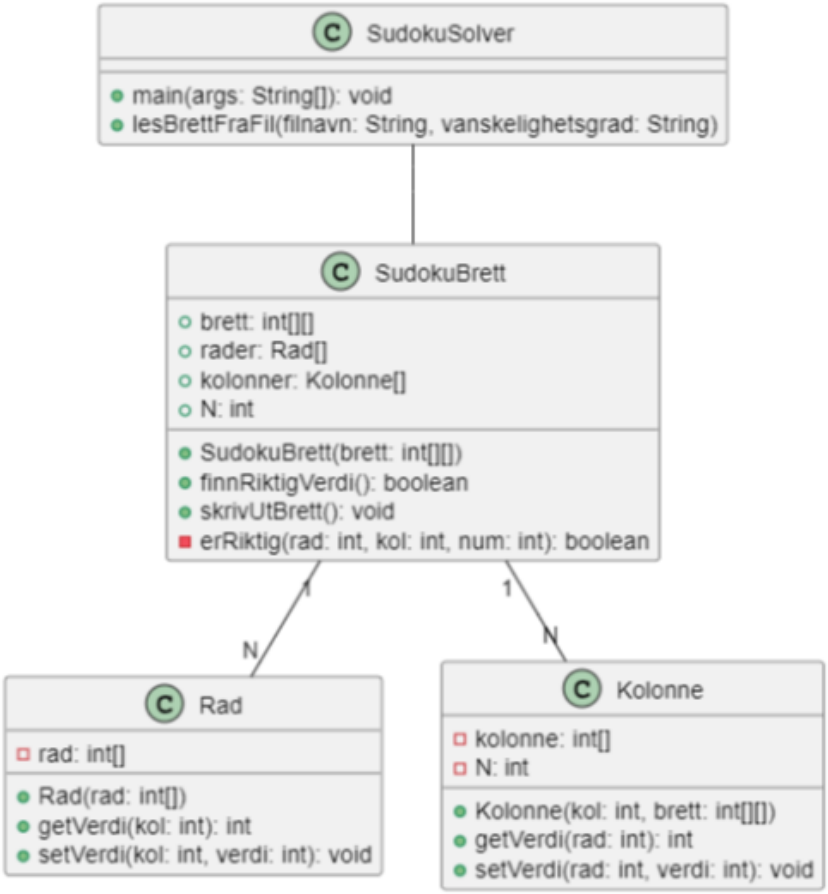
Løsning på datamaskin med en algoritme:
Det finnes flere algoritmer som kan løse sudoku: Sudoku solving algorithms Algoritmene kan beskrives matematisk (skriftlig), men for å teste dem er det enkleste å bruke en datamaskin med et program fremfor penn og papir.

Brute-force search: Backtracking
Dette er en algoritme som egentlig også bare "prøver og feiler" til den er ferdig med å løse problemet. I Sudoku kan dette gjøres ved å starte i øverste celle fra toppen (0,0), og traversere (løpe gjennom) alle celler fra topp til bunn (8,8). Problemet med dette er at det kan ende med et Sudoku brett som er uløselig eller ugyldig. For å sikre at programmet går tilbake og prøver et annet tall dersom en løsning ikke blir funnet på første gjennomløp, må vi implementere en backtracking-algoritme. Dette innebærer å prøve et tall (fra 1 til 9), og hvis det ikke fører til en løsning, gå tilbake og prøve neste tall. Demonstrasjon av backtracking-algoritme for Oblig3 er vist i forelesning, og video ligger på Canvas: Canvas-video

Oppgaver: Sudoku

Del 1:

Et 2D array skal brukes for å gi verdier til rad og kolonne objektene. Lag klasser for Rad og Kolonne med hver sin int array for verdiene. Alle datafelt skal være private i Rad-klassen og Kolonne-klassen. **Følgende UML må brukes, men retur typen for lesBrettFraFil er valgfri.**



Figur 2: UML diagram for klassene i oblig 3

NB Metodene i SudokuBrett skal implementeres i Del 3, men det kan være lurt å lage skrivUt-Brett som kan brukes for å sjekke/teste at Sudoku brett er lest inn korrekt. Det kan sjekkes ved å se på tallene i filen puzzle.dat og sammenligne med utskrift på skjerm fra metoden skrivUt-Brett.

Del 2:

Lag en Java-metode som leser inn en fil med Sudoku-brett og legger tallene i en 2D int-array. Filen ligger på Canvas og heter Puzzle.dat og er kodet binært. Brukeren kan velge vanskelighetsgrad med en hardkodet streng som:
“open”, “easy”, “medium” eller “hard”
Tips: Lag først et vanlig Java program med main-metode, der du leser inn tall inn i en int 2D array ut ifra hvilken vanskelighetsgrad som er valgt. Når du har fått til dette i main-metoden, må kode flyttes ut til en statisk metode som blir kalt opp i main. Retur type for metoden er valgfritt. Når du har lest inn tallene i en int 2D array, skal den brukes som parameter i konstruktør til SudokuBrett. Der skal Rad og Kolonne (klassene) brukes til å lagre tallene i hver sin int 1D array.

Del 3:

Bruk de to følgende algoritmene for å løse brettet:

```
finnRiktigVerdi:
    for rad fra 0 til N-1
        for kolonne fra 0 til N-1
            hvis brett[rad][kolonne] er lik 0
                for nummer fra 1 til N
                    hvis erRiktig(rad, kolonne, nummer) er sann
                        sett brett[rad][kolonne] til nummer

                        hvis finnRiktigVerdi() er sann
                            returner TRUE

                        // Backtracking -
                        // Tilbakestill verdien i cellen hvis ingen løsning finnes:
                        sett brett[rad][kolonne] til 0
                returner FALSE
    returner TRUE
```

Pseudokode for metoden finnRiktigVerdi, retur typen er boolean.

```
erRiktig(rad, kol, num)
    for x fra 0 til N-1
        hvis rader[rad].rad[x] er lik num ELLER
           kolonner[kol].kolonne[x] er lik num ELLER
           rader[rad - rad % 3 + x / 3].rad[kol - kol % 3 + x % 3] er lik num
            returner FALSE
    returner TRUE
```

Pseudokode for metoden erRiktig, retur typen er boolean.

Figur 3: Pseudo-kode for algoritmene i oblig 3

Begge metodene skal implementeres i klassen; SudokuBrett.

Del 4:

Når klassen; SudokuBrett er implementert etter UML på forrige side, skal den testes og du skal verifisere at den fungerer. Dette gjøres ved å lage objekt av SudokuBrett og kalle på dens metode finnRiktigVerdi. Dersom metoden finnRiktigVerdi returnerer true, er Sudoku spillet løst og du er ferdig med Oblig3. Tips: Bruk metoden skrivUtBrett for å se at løsning er korrekt.

Del 5:valgfri ekstra oppgave

Det er også mulig å test løsning ved å laste ned GUI modulen som bruker din løsning. GUI modulen kan koble til SudokuSolver. Dette krever refaktorering (endring) av koden som inkluderer din løsning. SudokuSolver inneholder ikke noe algoritme for å løsning av Sudoku. Testing løsning ved å kjøre GUI demoen og laste opp en oppgave-fil.

Del 6:

I denne oblig'en skal vi også bruke javadoc. Din java-fil med løsning, skal også ha oppsett for generering av JavaDoc. Når du er ferdig med java programmeringen skal følgende metoder dokumenteres med javadoc:

- lesBrettFraFil(...)
- finnRiktigVerdi()
- erRiktig(...)
- skrivUtBrett()

javadoc kan brukes ved å lime inn kommentar-blokken under her, før en metode i java-filen:

```
/**/**
* Hva gjør metoden og hvordan
* @param   Hvilken parameter tar den i signaturen
* @return       Hvilken verdi blir returnert dersom metoden har en retur type
*/
```

javadoc-kommandoen kan brukes på en terminal i samme katalog som java-filen ligger:
`javadoc -d docs -private -encoding UTF-8 -locale nb_NO Oblig3<studentnr>.java`
Dette vil generere en mappe som heter docs, der du kan åpne index.html for å se javadoc dokumentasjonen for java-filen din. Se følgende lenker for mer info om javadoc:
Oracle Javadoc Tool